Compiled spatial optimisation: exact resident state and GPU population search for landscape harvest scheduling

UBC FRESH Lab

November 3, 2026

Abstract

Spatially explicit forest planning models are difficult not because the objective is exotic, but because the model, the exact accounting state, and the search move are usually rebuilt together. This paper describes a compiled-state architecture that separates four concerns: a stable compiled model, compact candidate and sparse effect indexes, exact resident accounting, and batched GPU population search paired with exact CPU refinement. We evaluate the architecture on a public frozen TSA24 landscape with 208,255 operable blocks. A 1,000-generation GPU search reduces the weighted penalty from 1,722,144,291.336651 to 8,588,725.370498 in 9.381 s of search time; a 100,000-generation, five-seed campaign reaches 512.574–518.621 (mean 515.682) with exact CPU replay. The target uses deliberately unattainable harvest-level aspirations to pull the solution toward the maximum even-flow level; the final level is approximately 51.3–52.4M m^3 per 10-year period against a 70M aspiration. The frozen dataset, target, seed, and replay contract are public. We do not claim an independent model comparison or a state-of-the-art landscape result; the contribution is the architecture and its reproducibility contract.

1 Introduction

Forest planning models combine long planning horizons, spatially explicit treatment choices, and objectives that mix timber production with ecological and operational constraints. At the scale of a timber supply area, the decision is not a single harvest level but a schedule: which blocks receive which treatments, in which periods, under which spatial and temporal rules. The combinatorial state is large, the feasible set is structured, and the objective is usually a weighted compromise among targets. The resulting models are hard for exact methods and expensive for generic heuristics.

A recurring engineering problem sits underneath the optimisation problem. A compiled forest model contains blocks, fragments, strata, tracks, treatments, transitions, growth curves, products, and accounting rules. A search move changes one small part of that model, but the implementation often recomputes or reconstructs a much larger state to evaluate it. The model, the accounting state, and the move are therefore rebuilt together. This coupling makes experiments slow, makes replay fragile, and makes it difficult to separate the effect of a search method from the effect of an implementation detail.

This paper asks a narrower question than "which metaheuristic is best for forest planning?" It asks whether a compiled-state architecture can make a large landscape search exact, replayable, and fast enough to support controlled experiments. The architecture has five layers:

1. a compiled model;
2. a compact candidate index;
3. a sparse effect table over target cells;

4. exact resident accounting state;
5. a hybrid search loop that combines batched GPU population search with exact CPU refinement.

The first two layers make the model cheap to query. The sparse table makes an evaluation proportional to the cells a move touches rather than to the whole schedule. The resident state makes the objective authoritative and reversible. The hybrid loop uses the GPU for breadth and the CPU for exact accounting and local refinement.

We evaluate the architecture on a frozen TSA24/Prince George even-flow instance. The instance has 208,255 operable blocks out of 458,708 compiled blocks and a 20-period horizon. The target is a weighted goal-programming construction: a dominant even-flow target defines the shape, while deliberately unattainable harvest-level aspirations provide a persistent pull toward more volume. The realized level is an outcome, not a declared feasible floor. This construction is intentional; the 70M m^3 per-period aspiration is not a hard constraint and the reported residual is not evidence that the model is infeasible.

The paper makes three contributions:

- a compiled-state architecture that separates model, accounting, and search;
- an exact replay contract for a large landscape instance, including a public frozen artifact;
- a measured scaling record from 1,000 to 100,000 GPU generations, with five-seed evidence at the largest budget.

It does not make a competitive landscape claim. The CPU simulated-annealing comparison reported here is a weak, untuned secondary comparator, and no independent harvest-scheduling model is available on the same instance. Those limits are part of the result, not a footnote.

2 Related work

2.1 Spatial harvest scheduling

Spatial harvest scheduling has a long history in forest operations research. Lockwood and Moore [1993] formulated harvest scheduling with spatial constraints and applied simulated annealing to a problem in which adjacency and opening-size restrictions make the feasible set non-trivial. Their work established a template that remains recognizable: represent a schedule as a set of treatment decisions, define a penalty or objective over the resulting landscape, and use a stochastic search to move between feasible or penalized states.

Boston and Bettinger [2002] compared tabu search and a genetic algorithm on spatial harvest scheduling problems. Their comparison is important here because it treats the search method as an experimental factor rather than as an implementation detail. Bettinger et al. [2002] extended the comparison to eight heuristic planning techniques on three wildlife planning problems of increasing difficulty. Together, these studies show that the choice of neighbourhood, the representation of the schedule, and the handling of spatial constraints can matter as much as the name of the metaheuristic.

The present paper is not a replacement for those comparisons. It addresses the layer beneath them: how the model state and objective are maintained while a search runs. The architecture is designed to make that layer explicit, exact, and replayable, so that future comparisons can vary the search method without rebuilding the model and accounting machinery.

2.2 Exact state and neighbourhood search

Metaheuristic implementations often trade exactness for speed. A move is evaluated by an approximate delta, a cached value, or a full re-simulation; each choice changes what the search is actually optimising. For forest planning, this is a material issue because the objective is a weighted goal-programming penalty over multiple accounts and periods. An approximate evaluator can make a schedule look better than it is, and a full re-simulation can make a controlled experiment impractically slow.

The architecture described here keeps the exact objective as the authority. A move is priced by the cells it changes, but the resident state is updated with the same accounting rules used by the full evaluator. The search can therefore use a fast delta without giving up the exactness of the reported incumbent. This is closest in spirit to the exact incremental evaluation used in move-based local search, but it is applied to a compiled landscape model rather than to a generic permutation problem.

2.3 Reproducibility and software infrastructure

Reproducibility in applied optimisation is usually discussed at the level of random seeds and reported objective values. That is necessary but not sufficient. A result also depends on the compiled model, the candidate order, the effect table, the treatment definitions, the objective weights, the hardware, and the replay procedure. If those are not part of the artifact, a reader cannot tell whether a difference comes from the algorithm or from the data pipeline.

This paper treats the evidence record as part of the architecture. Each reported schedule is stored with its objective, seed, configuration, hardware, and a hash. An exact replay loads the schedule through the same model and evaluator, rather than trusting the search process that produced it. The frozen TSA24 artifact and its rebuild recipe are published so that the GPU result is not a local demonstration that cannot be inspected.

The related work on reproducible optimisation software is broad, and this paper does not attempt a general survey. Its narrower claim is that an exact resident state plus an explicit artifact can make a large landscape experiment auditable even when the search itself is stochastic.

3 Problem and model

3.1 Instance

The case study uses the TSA24/Prince George landscape compiled for WS4. The compiled dataset contains 458,708 blocks; 208,255 of them have at least one operable treatment chain and therefore form the search universe. The horizon is 200 years, divided into twenty 10-year accounting periods. The model includes blocks, fragments, tracks, strata, treatments, growth and yield curves, features, products, and account definitions. The schedule assigns a treatment chain to each operable block. An empty chain is a valid decision: a block may be left untreated.

The instance is frozen and published as `ws4-tsa24-evenflow-v1` with the compiled dataset, target file, initial schedule, license terms, and a checksum manifest. The artifact is a derived benchmark bundle; the raw BC source layers retain their own open-government terms.

3.2 Even-flow target and level aspirations

The objective is weighted goal programming. Each target contributes a one-sided penalty for falling below a minimum or exceeding a maximum:

$$s_{t,p} = \max(m_t - v_{t,p}, 0), \tag{1}$$
$$e_{t,p} = \max(v_{t,p} - M_t, 0), \tag{2}$$
$$\tau_{t,p} = w_t^- \, \phi(s_{t,p}) + w_t^+ \, \phi(e_{t,p}). \tag{3}$$

The shape is linear or quadratic. The target set is not a set of hard constraints. The names `minimum` and `maximum` name the hinge of the penalty, not a feasible interval.

The TSA24 target has two roles. First, one even-flow target spans periods 1–20, compares each period with the reference period, and charges a squared deviation outside a ±1% band with weight 10,000. Second, every period carries a linear harvest-level target with aspiration 70,000,000 m^3 per 10-year period and weight 100. The level aspiration is deliberately above the level the estate is expected to achieve. It is a pull-up device: every period has a downward gradient toward more harvest, including periods that would otherwise be left underused. The even-flow target selects the flattest profile at the level where the two roles reach their best weighted compromise. The realized level is therefore an outcome, not a declared floor.

This construction is the maximum-even-flow pattern documented for WS4. It is not an infeasibility error, and a large residual on the level term is not evidence that the target should be removed. It is the expected consequence of an aspirational target that creates a search gradient. The paper reports the split between the flow and level terms, the realized harvest level, and the target values so that a reader can inspect the compromise rather than infer it from a single total.

3.3 Initial schedule and feasibility

The initial schedule is the oldest-first seed from the public artifact. It is a valid block/treatment schedule and therefore a feasible starting point for the search, but it is not an optimized control. The initial objective is 1,722,144,291.336651. The very large initial penalty is dominated by even-flow deviations; the seed is a starting point chosen for reproducibility, not a baseline chosen to make the search look effective.

The model has no hard spatial constraints in this run. The target residuals are soft, and the reported schedule is a compromise. The paper therefore distinguishes schedule validity from target satisfaction: the schedule is feasible, while the targets are aspirations.

3.4 Objective and evaluation

For a schedule `S`, let `v(S, p, a)` be the realized value of account `a` in period `p`. The objective is

$$Z(S) = \sum_t \sum_{p \in P_t} \tau_{t,p}\big(v_{t,p}(S)\big). \tag{4}$$

The implementation evaluates this objective exactly. A search move may use a cached per-block contribution to price its effect, but the resident state is updated with the same accounting rules as the full evaluator. The reported objective is never an approximation produced by the search loop. This distinction is central to the evidence contract: a fast delta is allowed, but the incumbent must replay through the exact evaluator.

4 Architecture

4.1 Compiled model

The compiled dataset is the boundary between ingestion and execution. It is a versioned directory of columnar tables plus a manifest. The tables describe fragments, blocks, tracks, strata, treatments, curves, features, products, accounts, groups, and compile diagnostics. The manifest records the schema version, source identities and hashes, compile options, and table list.

Compilation happens at the fragment level; a block is a runtime aggregate of its fragments. The separation matters because fragments carry the inventory attributes, while the search and reporting interfaces operate on blocks. The search never reads the raw inventory or recompiles the recipe.

4.2 Candidate index

For each block, the candidate index stores the feasible treatment chains in a stable order. A chain is an ordered sequence of treatment events `(treatment, year)`. The order is part of the contract: candidate indices are referenced by the effect table and by the search, so changing the order changes the meaning of a cached effect.

The index is built by walking track transitions from the block's initial state and applying the treatment eligibility and operability windows. It includes the empty chain. The index is deliberately smaller than the full cross-product of treatments and years, but it remains richer than a single "harvest/no harvest" decision. The search can toggle a chain, shift an event, swap chains between blocks, or apply a larger destroy-and-repair move.

4.3 Sparse effect table

A compiled landscape has many blocks and many target cells, but a single block/chain pair affects only a small number of `(year, account)` cells. The effect table stores, for each block and candidate chain, a sparse row of those cells and their values relative to the untreated block. The table also stores the empty-schedule base totals and block row starts.

The representation makes evaluation local. A move on one block is priced by the cells that block's old and new chains touch, not by iterating over the whole schedule. The sparse rows are built once and cached; the table identity includes the dataset fingerprint, candidate order, target-cell geometry, horizon, period width, and table format version. A change to weights, priorities, or targets is a pricing change, not a table rebuild.

4.4 Resident exact state

The optimizer maintains a resident state of realized totals per `(year, account)`, together with the current chain assigned to each block. Applying a move adds the new chain's contribution and subtracts the old chain's contribution. Undo restores the previous state exactly. The state is authoritative: the search uses it for the objective, and the replay path reconstructs it from the schedule through the same evaluator.

For a move on block `b` from chain `old` to chain `new`, the objective delta is

$$\Delta Z_b(c_{\text{old}} \to c_{\text{new}}) = \sum_{(y,a)\in C_b} \left[\pi_{y,a}(T_{y,a} - o_{y,a} + n_{y,a}) - \pi_{y,a}(T_{y,a})\right]. \tag{5}$$

Because the penalty is separable across cells, this delta is exact. The cost is proportional to the number of cells the block touches, not to the number of blocks in the schedule. The implementation uses this identity in the local-search paths and retains a whole-schedule evaluator as a reference.

4.5 GPU population search and CPU refinement

The architecture separates breadth from exactness. A batched GPU population evaluates candidate-index genomes against the sparse effect table. The GPU path is used for population-scale exploration; it does not replace the exact accounting state. The CPU path loads an elite schedule into the exact resident state, applies local refinement moves, updates the objective by exact deltas, and writes the incumbent back as a GPU seed.

The handoff is therefore:

$$\begin{aligned} \text{GPU population} &\longrightarrow \text{elite schedule} \\ &\longrightarrow \text{exact CPU state load} \\ &\longrightarrow \text{exact local refinement} \\ &\longrightarrow \text{exact replay} \\ &\longrightarrow \text{refined elite as GPU seed} \end{aligned}$$

This division is an engineering choice, not a claim that GPU search is universally better than CPU search. It allows the population to explore many candidate schedules while the exact evaluator remains the authority for the objective and the final schedule.

4.6 Replay and provenance

Every reported schedule is stored as a complete block/treatment/offset assignment. The record carries the objective, seed, configuration, hardware, software, schedule path, and SHA-256. Replay loads the schedule through the compiled model and exact evaluator and compares the resulting objective with the record. A record with a mismatched hash or objective is not evidence.

The architecture therefore has two levels of reproducibility. The search is reproducible from its configuration and seed for a fixed implementation, and the reported result is independently replayable from the stored schedule even if the search process is not rerun. The second property is the one the paper relies on for its tables.

5 Experimental design

6 Experimental design

This section will define the hardware, seeds, budgets, stopping rules, baselines, and metrics. The CPU-SA baseline will be labelled as an untuned secondary comparator, not a controlled performance proof.

7 Results

This section will report the 1k/10k/100k scaling curve, the five-seed 100k distribution, the flow/level residual decomposition, the realized harvest profile, and the architecture ablation. It will not describe the 517.86 penalty as “near zero.”

8 Discussion

This section will separate what the architecture makes possible from what the current evidence does not establish: independent model performance, operational impact, and global optimality.

9 Limitations and conclusion

The TSA24 instance is a public frozen artifact, but there is no independent model baseline. The 100,000-generation result is a maximum even-flow compromise, not a proof of the exact maximum. The CPU-SA comparator is weak and time-limited. The architecture is the contribution; the competitive landscape claim remains open.

Data and code availability

The TSA24 benchmark artifact is available at `https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1`. The build and replay recipe is in `BENCHMARK_ARTIFACT.md` in that repository.

References

Pete Bettinger, David Graetz, John Sessions, et al. Eight heuristic planning techniques applied to three increasingly difficult wildlife planning problems. *Silva Fennica*, 36, 2002. doi: 10.14214/sf.545.

Kevin Boston and Pete Bettinger. Combining tabu search and genetic algorithm heuristic techniques to solve spatial harvest scheduling problems. *Forest Science*, 48(1):35–46, 2002. doi: 10.1093/forestscience/48.1.35.

Christopher Lockwood and Thomas Moore. Harvest scheduling with spatial constraints: a simulated annealing approach. *Canadian Journal of Forest Research*, 23:468–478, 1993. doi: 10.1139/x93-065.